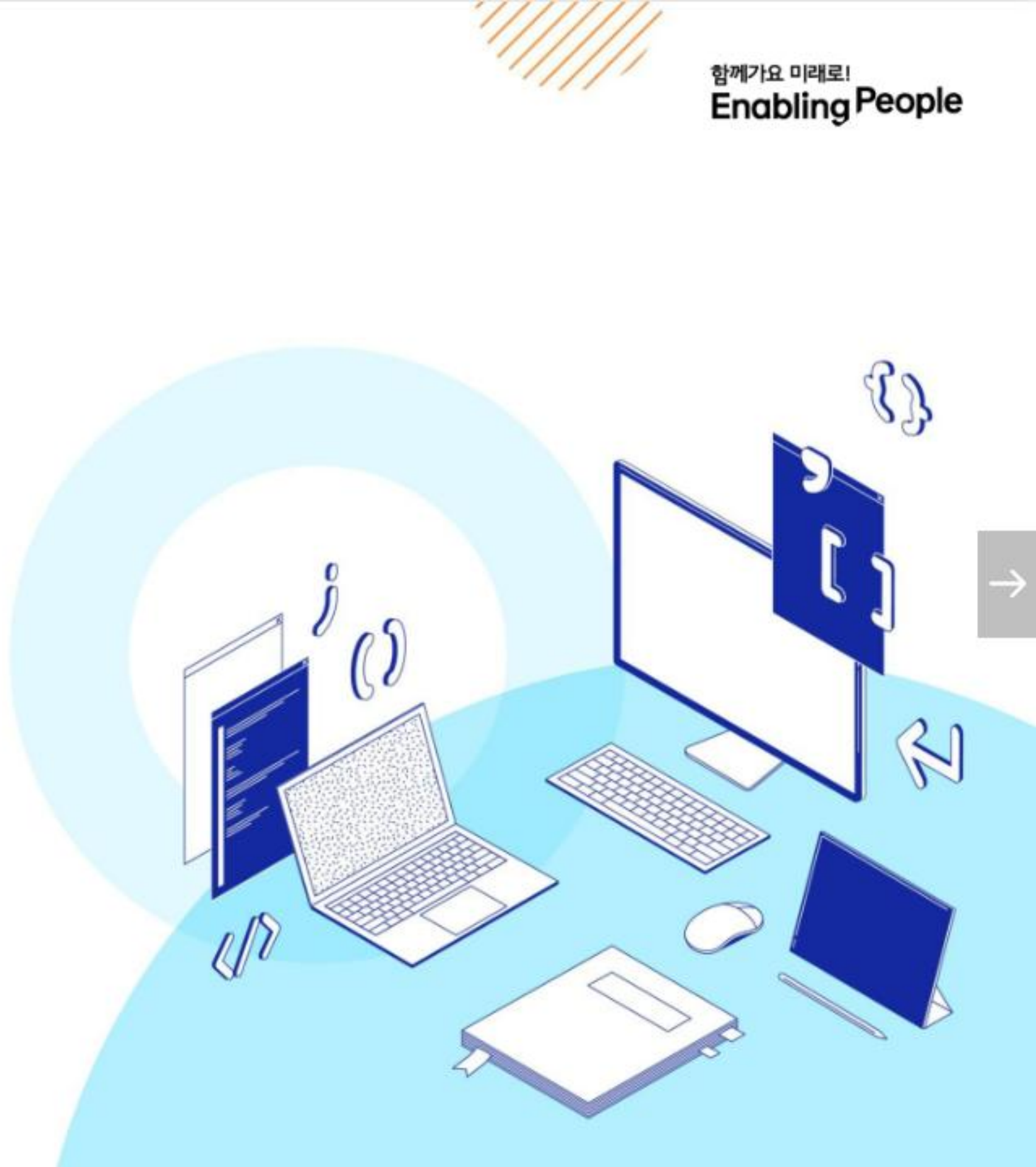


SSAFY

Javascript

Basic syntax 02

Python



목 차

객체

- 구조 및 속성
- 메서드
- this
- 추가 객체 문법
- JSON

배열

- 배열 메서드

목 차

Array helper method

- 콜백 함수
- forEach
- map
- 배열 순회 종합
- 배열 with '전개 구문'

참고

- 클래스
- 콜백 함수의 이점
- forEach에서 break 사용하기
- 배열은 객체다

"여러 사람의 데이터를 다루려면 무엇부터 생각해야 할까요?"

1. 먼저 한 사람의 이름, 나이, 직업을 어떻게 묶을 수 있을까요? ()
2. 여러 사람의 데이터를 목록으로 관리하려면 어떻게 해야 할까요? ()
3. 목록의 모든 사람에게 똑같은 작업을 시키려면 어떻게 해야 할까요? ()

- 이 질문들의 답을 찾아가며 자바스크립트로 데이터를 효율적으로 다루는 법을 배웁니다.

학습 목표

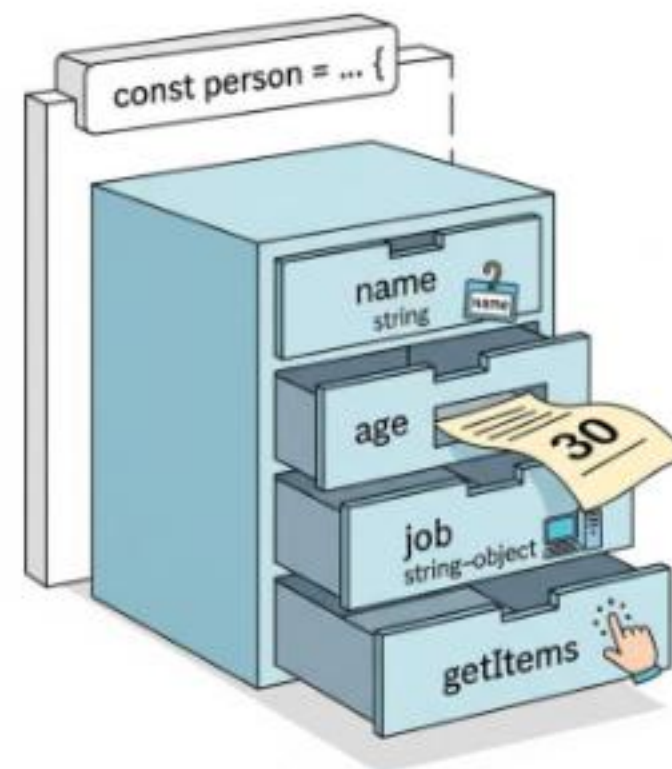
- ✓ 객체의 메서드 안에서 `this`를 사용해 속성에 접근한다.
- ✓ 화살표 함수에서 `this`가 상위 스코프를 참조함을 안다.
- ✓ 구조 분해 할당을 사용해 객체의 속성을 변수로 할당한다.
- ✓ `JSON.stringify`와 `JSON.parse`로 객체와 JSON을 변환한다.
- ✓ 콜백 함수를 이해하고 다른 함수의 인자로 전달할 수 있다.
- ✓ `forEach`와 `map` 메서드의 차이를 알고 배열을 순회한다.
- ✓ 전개 구문을 활용하여 원본 배열을 수정하지 않고 합친다.

객체

Object

Object

키로 구분된 데이터 집합을 저장하는 자료형
(data collection)



〈그림1_Javascript_Basic syntax 02_Object 예시〉

구조 및 속성

객체 구조

- 중괄호('{}')를 이용해 작성
- 중괄호 안에는 key: value 쌍으로 구성된 속성(property)를 여러 개 작성 가능
- key는 문자형만 허용
- value는 모든 자료형 허용

```
const user = {  
  name: 'Alice',  
  'key with space': true,  
  greeting: function () {  
    return 'hello'  
  }  
}
```

속성 참조

- 점('.') 표기법 또는 대괄호('[]') 표기법으로 객체 속성에 접근
- key 이름에 띄어쓰기 같은 구분자가 있으면 대괄호 접근만 가능

```
// 조회
console.log(user.name) // Alice
console.log(user['key with space']) // true

// 추가
user.address = 'korea'
console.log(user) // {name: 'Alice', key with space: true, address: 'korea', greeting: f}

// 수정
user.name = 'Bella'
console.log(user.name) // Bella

// 삭제
delete user.name
console.log(user) // {key with space: true, address: 'korea', greeting: f}
```

'in' 연산자

- 속성이 객체에 존재하는지 여부를 확인
- 객체의 키나 배열의 인덱스 존재 여부를 확인하는 연산자

```
console.log('greeting' in user) // true  
console.log('country' in user) // false
```



프로토타입 :

객체들이 기능을 물려받는 원본,
즉 '부모' 역할을 하는 객체

<개념1_Javascript_Basic syntax 02_프로토타입>



프로토타입 체인:

자신에게 없는 속성이나 기능을
부모, 조상 순으로 찾아가는 것

<개념2_Javascript_Basic syntax 02_프로토타입 체인>

TIP

- 객체에서 값의 포함 여부를 확인하려면, "in" 연산자 대신 "hasOwnProperty()" 메서드를 사용하는 것이 올바릅니다.
- 프로토타입 체인을 따라 상속된 속성까지 확인하므로, 의도치 않게 true가 나올 수 있어 주의해야 합니다.

메서드

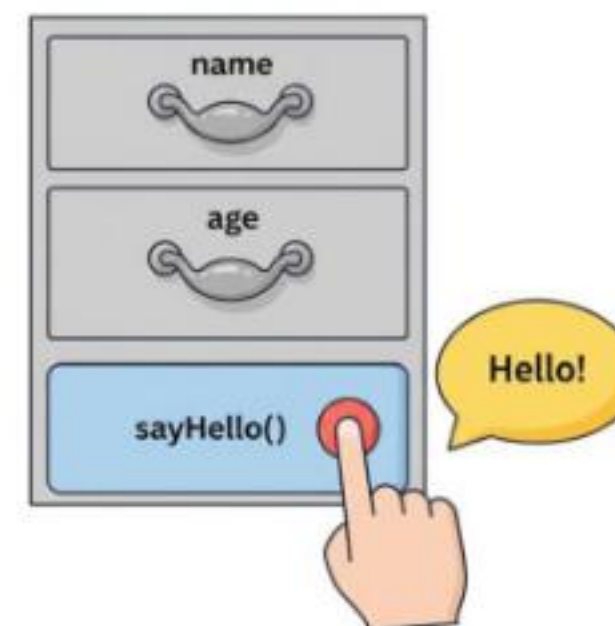
Method

Method

객체 속성에 정의된 함수

object.method() 방식으로 호출
메서드는 객체가 '행동' 할 수 있게 함

JavaScript Object



<그림2_Javascript_Basic syntax 02_Javascript Method 예시>

```
console.log(user.greeting()) // hello
```


Method 기본 문법

- 메서드도 값이 함수인 속성

```
const myObj2 = {  
  numbers: [1, 2, 3],  
  myFunc: function () {  
    this.numbers.forEach(function (number) {  
      console.log(this) // window  
    })  
  }  
}  
  
console.log(myObj2.myFunc())
```

- 메서드와 일반 함수의 차이는?
 - 메서드는 자신이 속한 객체의 다른 속성들에 접근할 수 있음
 - 이를 위한 방법이 this

this

● 'this' keyword (1/2)

Method

객체 속성에 정의된 함수

'this' 키워드를 사용해 객체 자신의 속성이나 메서드에 접근하여
특정 작업을 수행 할 수 있음

'this' keyword (2/2)

this

함수나 메서드를 호출한 객체를 가리키는 키워드

➤ 'this' 키워드를 사용해
객체에 대한 특정한 작업을 수행 할 수 있음



〈그림3_Javascript_Basic syntax 02_this 예시〉

Method & this 사용 예시

```
<!-- this-keyword.html -->
```

```
const person = {  
  name: 'Alice',  
  greeting: function () {  
    return `Hello my name is ${this.name}`  
  },  
}
```

```
console.log(person.greeting()) // Hello my name is Alice
```


JavaScript에서 this는 함수를 “호출하는 방법”에 따라 가리키는 대상이 달라짐

호출 방법	대상
일반 함수에서의 단순 호출	전역 객체
객체에서의 메서드 호출	메서드를 호출한 객체

<표1_Javascript_Basic syntax 02_this 호출 방법에 따른 대상>

단순 호출 this

- 가리키는 대상 $\Rightarrow$ 전역 객체

```
<!-- this-keyword.html -->
```

```
const myFunc = function () {  
  return this  
}
```

```
console.log(myFunc()) // window
```

메서드 호출 시 this

- 가리키는 대상 $\Rightarrow$ 메서드를 호출한 객체

```
<!-- this-keyword.html -->
```

```
const myObj = {  
  data: 1,  
  myFunc: function () {  
    return this  
  }  
}
```

```
console.log(myObj.myFunc()) // myObj
```

중첩된 함수에서의 this 문제점

- forEach의 인자로 전달된 콜백 함수는 일반 함수로 호출되므로, this는 전역 객체를 가리킴

```
const myObj2 = {  
  numbers: [1, 2, 3],  
  myFunc: function () {  
    this.numbers.forEach(function (number) {  
      console.log(this) // window  
    })  
  }  
}  
  
console.log(myObj2.myFunc())
```

중첩된 함수에서의 this 문제점

- forEach의 인자로 작성된 함수는 일반적인 함수 호출이기 때문에 this가 전역 객체를 가리킴

```
const myObj2 = {  
  numbers: [1, 2, 3],  
  myFunc: function () {  
    this.numbers.forEach(function (number)  
    {  
      console.log(this) // window  
    })  
  }  
}  
  
console.log(myObj2.myFunc())
```



해결책

- 화살표 함수는 자신만의 this를 가지지 않음
- 따라서 외부 함수(myFunc)에서의 this 값을 가져옴

```
const myObj3 = {  
  numbers: [1, 2, 3],  
  myFunc: function () {  
    this.numbers.forEach((number) =>  
    {  
      console.log(this) // myObj3  
    })  
  }  
}  
  
console.log(myObj3.myFunc())
```


JavaScript 'this' 정리

- JavaScript의 함수는 호출될 때 this를 암묵적으로 전달 받음
- JavaScript에서 this는 함수가 “호출되는 방식”에 따라 결정되는 현재 객체를 나타냄
- Python의 self와 Java의 this가 선언 시점에 이미 값이 정해지는 것과 달리

JavaScript의 this는 **함수가 호출될 때 동적으로 결정**

- 장점
 - 함수(메서드)를 하나만 만들어 여러 객체가 공유하여 각자 자신의 데이터로 동작하게 할 수 있음
- 단점
 - 이런 유연함이 실수로 이어질 수 있다는 것

TIP

- 개발자는 this 의 동작 방식을 충분히 이해하고 장점을 취하면서 실수를 피하는 데에 집중해야 합니다.
- this 가 헛갈릴 땐 '누가 점(.)을 찍어 호출했는가?' 에 집중해보세요. 점 앞의 객체가 this가 됩니다.

추가 객체 문법

추가 객체 문법

1. 단축 속성
2. 단축 메서드
3. 계산된 속성(computed property name)
4. 구조 분해 할당 (destructuring assignment)
5. 객체와 전개 구문 (Spread Syntax)
6. Object keys() / values() / entries()
7. Optional chaining ('?.')

1. 단축 속성

- 키 이름과 값으로 쓰이는 변수의 이름이 같은 경우 단축 구문을 사용할 수 있음

```
const name = 'Alice'  
const age = 30  
const user = {  
  name: name,  
  age: age,  
}
```



```
const name = 'Alice'  
const age = 30  
const user = {  
  name,  
  age,  
}
```

2. 단축 메서드

- 메서드 선언 시 function 키워드 생략 가능

```
const myObj1 = {  
  myFunc: function () {  
    return 'Hello'  
  }  
}
```



```
const myObj1 = {  
  myFunc() {  
    return 'Hello'  
  }  
}
```

3. 계산된 속성 (computed property name)

- 키가 대괄호([])로 둘러싸여 있는 속성
- 고정된 값이 아닌 변수 값을 사용할 수 있음

```
const product = prompt('물건 이름을 입력해주세요')
const prefix = 'my'
const suffix = 'property'
const bag = {
  [product]: 5,
  [prefix + suffix]: 'value',
}
console.log(bag) // {연필: 5, myproperty: 'value'}
```

TIP

- 대괄호 안의 표현식이 너무 복잡해지면, 어떤 키가 생성될 지 파악하기 어려워 가독성이 떨어질 수 있습니다.
- 동적으로 키를 만들다 보면 의도치 않게 같은 이름의 키가 생성되어, 기존 값이 덮어써질 위험이 있습니다.

4. 구조 분해 할당 (destructuring assignment) (1/2)

- 배열 또는 객체를 분해하여 객체 속성을 변수에 쉽게 할당할 수 있는 문법

```
const userInfo = {  
  firstName: 'Alice',  
  userId: 'alice123',  
  email: 'alice123@gmail.com'  
}  
  
const firstName = userInfo.firstname  
const userId = userInfo.userId  
const email = userInfo.email
```



```
const userInfo = {  
  firstName: 'Alice',  
  userId: 'alice123',  
  email: 'alice123@gmail.com'  
}  
  
const { firstName } = userInfo  
const { firstName, userId } = userInfo  
const { firstName, userId, email } = userInfo  
  
// Alice alice123 alice123@gmail.com  
console.log(firstName, userId, email)
```


4. 구조 분해 할당 (destructuring assignment) (2/2)

- '함수의 매개변수'로 객체 구조 분해 할당 활용 가능

```
const person = {  
  name: 'Bob',  
  age: 35,  
  city: 'London',  
}  
  
function printInfo({ name, age, city }) {  
  console.log(`이름: ${name}, 나이: ${age}, 도시: ${city}`)  
}  
  
// 함수 호출 시 객체를 구조 분해하여 함수의 매개변수로 전달  
printInfo(person) // 이름: Bob, 나이: 35, 도시: London
```

5. 객체와 전개 구문 (Spread Syntax, ...)

- “객체 복사”
 - 객체 내부에서 객체 전개
- 얇은 복사에 활용 가능

```
const obj = {b: 2, c: 3, d: 4}
const newObj = {a: 1, ...obj, e: 5}
console.log(newObj) // {a: 1, b: 2, c: 3, d: 4, e: 5}
```



얇은 복사 :
겉(최상위 속성)만 복사하고,
속(중첩 객체)은 공유하는 복사

<개념3_Javascript_Basic syntax 02_얇은 복사>

6. 유용한 객체 메서드

- Object.keys()
 - Object 의 Key 값들을 리스트로 반환
- Object.values()
 - Object 의 Value 값들을 리스트로 반환
- Object.entries()
 - Object 의 key와 Value 값들을 한 쌍으로 묶은 리스트로 반환

```
const profile = {  
  name: 'Alice',  
  age: 30,  
}  
// ['name', 'age']  
console.log(Object.keys(profile))  
  
// ['Alice', 30]  
console.log(Object.values(profile))  
// [['name', 'Alice'], ['age', 30]]  
console.log(Object.entries(profile))
```


7. Optional chaining ('?.')

- 속성이 없는 중첩 객체에 접근하려 할 때 에러 발생 없이 안전하게 접근하는 방법
- 만약 참조 대상이 null 또는 undefined라면 에러가 발생하는 것 대신 평가를 멈추고 undefined를 반환

```
const user = {  
  name: 'Alice',  
  greeting: function () {  
    return 'hello'  
  }  
}  
  
console.log(user.address.street) // Uncaught TypeError  
console.log(user.address?.street) // undefined  
  
console.log(user.nonMethod()) // Uncaught TypeError  
console.log(user.nonMethod?.()) // undefined
```


7. Optional chaining ('?.') 장점

- 참조가 누락될 가능성이 있는 경우 연결된 속성으로 접근할 때 더 짧고 간단한 표현식을 작성할 수 있음
- 어떤 속성이 필요한지에 대한 보증이 확실하지 않는 경우에 객체의 내용을 보다 편리하게 탐색할 수 있음
- 만약 Optional chaining을 사용하지 않는다면 다음과 같이 '&&' 연산자를 사용해야 함

```
const user = {  
  name: 'Alice',  
  greeting: function () {  
    return 'hello'  
  }  
}  
  
console.log(user.address && user.address.street) // undefined
```

7. Optional chaining ('?.') 주의사항

1. Optional chaining은 존재하지 않아도 괜찮은 대상에만 사용해야 함 (남용 X)

- 왼쪽 평가대상이 없어도 괜찮은 경우에만 선택적으로 사용
- 중첩 객체를 에러 없이 접근하는 것이 사용 목적이기 때문

```
// 이전 예시 코드에서 user 객체는 논리상 반드시 있어야 하지만 address는 필수 값이 아님  
// user에 값을 할당하지 않은 문제가 있을 때 바로 알아낼 수 있어야 하기 때문
```

```
// Bad  
user?.address?.street  
// Good  
user.address?.street
```

2. Optional chaining 앞의 변수는 반드시 선언되어 있어야 함

```
console.log(myObj?.address) // Uncaught ReferenceError: myObj is not defined
```

7. Optional chaining ('?.') 정리

1. obj?.prop

- obj가 존재하면 obj.prop을 반환하고, 그렇지 않으면 undefined를 반환

2. obj?.[prop]

- obj가 존재하면 obj[prop]을 반환하고, 그렇지 않으면 undefined를 반환

3. obj?.method()

- obj가 존재하면 obj.method()를 호출하고, 그렇지 않으면 undefined를 반환

TIP

- null과 undefined일 때만 동작합니다.
- 체인 중간이 null인 경우, 그 뒤의 코드는 실행되지 않습니다. 이를 '단락 평가'라고 부릅니다.

JSON

JSON

JavaScript Object Notation

Key-Value 형태로 이루어진 자료 표기법

- JavaScript의 Object와 유사한 구조를 가지고 있지만 JSON은 일정한 형식을 가진 “문자열” 입니다.
- JavaScript에서 JSON을 사용하기 위해서는 Object 자료형으로 변경해야 합니다.
- 특정 언어에 종속되지 않는 데이터 형식으로, API 통신 등에서 널리 사용됩니다.

Object ⇨ JSON

- JSON.stringify() 를 사용해 객체를 문자열로 변환

```
const jsObject = {  
  coffee: 'Americano',  
  iceCream: 'Cookie and cream',  
}
```

```
// Object -> JSON  
const objToJson = JSON.stringify(jsObject)  
  
// {"coffee":"Americano","iceCream":"Cookie and cream"}  
console.log(objToJson)  
  
// string  
console.log(typeof objToJson)
```

JSON ⇨ Object

- JSON.parse()를 사용해 문자열을 객체로 변환

```
// JSON -> Object  
const jsonToObj = JSON.parse(objToJson)  
  
// { coffee: 'Americano', iceCream: 'Cookie and cream' }  
console.log(jsonToObj)  
  
// object  
console.log(typeof jsonToObj)
```

배열

배열

Array

순서가 있는 데이터 집합을 저장하는
자료구조

- 객체는 키(key)로 데이터를 관리하지만, 순서가 중요하지 않습니다.
- '첫 번째', '두 번째'처럼 순서가 중요한 데이터 묶음이 필요할 때 사용하는 것이 바로 “순서가 있는 컬렉션, 배열(Array)”입니다.
- 하지만 배열의 인덱스는 숫자로만 이루어져 있어, “키 자체가 데이터의 의미를 설명해주지 못하고”, 특정 값을 찾기 위해서는 배열의 모든 요소를 “처음부터 순서대로 확인”해야 하는 단점이 있습니다.

배열 구조

- 대괄호('[]')를 이용해 작성
- 요소의 자료형은 제약 없음
- length 속성을 사용해 배열에 담긴 요소 개수 확인 가능

```
const names = ['Alice', 'Bella', 'Cathy']
```

```
console.log(names[0]) // Alice
```

```
console.log(names[1]) // Bella
```

```
console.log(names[2]) // Cathy
```

```
console.log(names.length) // 3
```

배열 메서드

배열 주요 메서드

- push()
- pop()
- unshift()
- shift()

push()

- 배열 끝에 요소를 추가
- 원본 배열을 직접 수정
- 반환 값: 추가된 후의 새로운 배열의 길이

```
const names = ['Alice', 'Bella', 'Cathy']
```

```
names.push('Dan')
```

```
// ['Alice', 'Bella', 'Cathy', 'Dan']  
console.log(names)
```

pop()

- 배열 끝 요소를 제거
- 원본 배열을 직접 수정
- 반환 값: 제거한 요소

```
const names = ['Alice', 'Bella', 'Cathy']
```

```
console.log(names.pop()) // Dan
```

```
// ['Alice', 'Bella', 'Cathy']  
console.log(names)
```


unshift()

- 배열 앞에 요소를 추가
- 원본 배열을 직접 수정
- 반환 값: 추가된 후의 새로운 배열의 길이
- 배열의 모든 요소를 뒤로 한 칸씩 밀어야 하므로, 배열이 클수록 성능이 저하 (가급적 사용 X)

```
names.unshift('Eric')
```

```
// ['Eric', 'Alice', 'Bella', 'Cathy']  
console.log(names)
```

shift()

- 배열 앞 요소를 제거하고, 제거한 요소를 반환
- 원본 배열을 직접 수정
- 반환 값: 제거한 요소
- 배열의 모든 요소를 당겨와야 하므로, 배열이 클수록 성능이 저하 (가급적 사용 X)

```
console.log(names.shift()) // Eric
```

```
// ['Alice', 'Bella', 'Cathy']  
console.log(names)
```

Array helper method

Array Helper Methods

- 배열 조작을 보다 쉽게 수행할 수 있는 특별한 메서드 모음
- ES6에 도입
- 배열의 각 요소를 **순회**하며 각 요소에 대해 함수(**콜백함수**)를 호출
- 대표 메서드
 - forEach(), map()
 - filter()
 - every()
 - some()
 - reduce()
 - 등등
- 메서드 호출 시 인자로 함수(**콜백함수**)를 받는 것이 특징

콜백 함수

콜백 함수

콜백 함수

Callback function

다른 함수에 인자로 전달되는 함수

- 외부 함수 내에서 호출되어 일종의 루틴이나 특정 작업을 진행
- 특정 작업(1초 기다리기, ...)이 완료된 후, 시스템에 의해 나중에 호출(call back) 되는 함수

콜백 함수 예시 1

```
const numbers1 = [1, 2, 3]

numbers1.forEach(
  function (num) {
    console.log(num)
  }
)

// 1
// 2
// 3
```

콜백 함수 예시 2

```
const numbers2 = [1, 2, 3]

const callBackFunction = function (num) {
  console.log(num)
}

numbers2.forEach(callBackFunction)

// 1
// 2
// 3
```

주요 Array Helper Methods

- forEach
 - 배열 내의 모든 요소 각각에 대해 함수(콜백함수)를 호출
 - 반환 값 없음
- map
 - 배열 내의 모든 요소 각각에 대해 함수(콜백함수)를 호출
 - 함수 호출 결과를 모아 새로운 배열을 반환

forEach

forEach()

- 배열의 각 요소를 반복하며 모든 요소에 대해 함수(콜백함수)를 호출
- 구조

```
// forEach 구문  
arr.forEach(callback(item[, index[, array]]))
```

```
// forEach 예  
array.forEach(function (item, index, array) {  
    // do something  
})
```

- 콜백함수는 3가지 매개변수로 구성
 - item : 처리할 배열의 요소
 - index : 처리할 배열 요소의 인덱스 (선택 인자)
 - array : forEach를 호출한 배열 (선택 인자)
- 반환 값
 - undefined

forEach 예시

- 동일한 결과를 만들어 냄
- 간단한 콜백 함수의 경우, 화살표 함수를 사용하는 것이 가독성, 'this'를 다루는 방식의 차이가 있으므로 가능한 **화살표 함수** 사용이 권장

```
// 실행 코드
const names = ['Alice', 'Bella', 'Cathy']

// 일반 함수 표기
names.forEach(function (name) {
  console.log(name)
})

// 화살표 함수 표기
names.forEach((name) => {
  console.log(name)
})
```

```
// 출력 결과
```

```
Alice
Bella
Cathy
```

```
Alice
Bella
Cathy
```

forEach 활용

- forEach는 항상 undefined를 반환
- break 문으로 반복을 중단할 수 없음 ※ 반복을 중단하는 방법은 이후 “참고”에서 진행
- 간결한 코드를 위해서 필요한 매개변수만 활용

// 실행 코드

```
const names = [ 'Alice', 'Bella', 'Cathy' ]
```

```
names.forEach((name, index, array) => {  
  console.log(`${name} / ${index} / ${array}`)  
})
```

// 출력 결과

```
Alice / 0 / Alice, Bella, Cathy
```

```
Bella / 1 / Alice, Bella, Cathy
```

```
Cathy / 2 / Alice, Bella, Cathy
```

map

map()

- 배열의 모든 요소에 대해 함수(콜백 함수)를 호출하고, 반환 된 호출 결과 값을 모아 새로운 배열을 반환
- 구조

```
// map 구문  
arr.map(callback(item[, index[, array]]))
```

```
const newArr = array.map(function (item, index, array) {  
  // do something  
})
```

- forEach의 매개 변수와 동일
- 반환 값
 - 배열의 각 요소에 대해 실행한 “callback의 결과를 모은 새로운 배열”
 - forEach 동작 원리와 같지만 forEach와 달리 새로운 배열을 반환함

map 예시

- 배열을 순회하며 각 객체의 name 속성 값을 추출하기 (for...of 와 비교)
- map()은 '배열 반환' 이라는 의도가 명확히 나타나, for 문보다 코드가 간결하고 직관적
- map()은 새로운 배열을 반환하므로, 다른 메서드를 체이닝할 수 있음



체이닝 :

함수의 반환 값에, 꼬리를 물고
다음 함수를 바로 호출하는 기술.

<개념4_Javascript_Basic syntax 02_체이닝>

```
const persons = [
  { name: 'Alice', age: 20 },
  { name: 'Bella', age: 21 }
]

let result1 = []
for (const person of persons) {
  result1.push(person.name)
}

console.log(result1) // ['Alice', 'Bella']
```

```
const persons = [
  { name: 'Alice', age: 20 },
  { name: 'Bella', age: 21 }
]

const result2 = persons.map(function (person) {
  return person.name
})

console.log(result2) // ['Alice', 'Bella']
```

map 활용 (1/2)

- 화살표 함수를 활용해 간결하게 활용할 수 있음
- 원본 배열(names)를 변경하지 않고, 항상 새로운 배열을 반환(불변성)

```
const names = ['Alice', 'Bella', 'Cathy']

const result3 = names.map(function (name) {
  return name.length
})

const result4 = names.map((name) => {
  return name.length
})

console.log(names) // ['Alice', 'Bella', 'Cathy']
console.log(result3) // [5, 5, 5]
console.log(result4) // [5, 5, 5]
```


map 활용 (2/2)

- 커스텀 콜백 함수 활용
 - 콜백 함수를 변수에 담아두면, map 외 다른 곳에서도 같은 로직을 할 수 있어 유용
- myCallbackFunc()가 아닌 **myCallbackFunc** 를 전달

```
const numbers = [1, 2, 3]

const myCallbackFunc = function (number) {
  return number * 2
}

const doubleNumber = numbers.map(myCallbackFunc)

console.log(doubleNumber) // [2, 4, 6]
```


Javascript - map

- map 메서드에 callBackFunc 함수를 인자로 넘겨 numbers 배열의 각 요소를 callBackFunc 함수의 인자로 사용

```
const numbers = [1, 2, 3]

const callBackFunction = function (number) {
  return number ** 2
}

const newNumbers = numbers.map(callBackFunction)
```

Python - map

- python의 map에 square 함수를 인자로 넘겨 numbers 배열의 각 요소를 square 함수의 인자로 사용

```
numbers = [1, 2, 3]

def square(num):
    return num ** 2

new_numbers = list(map(square, numbers))
```

배열 순회 종합

배열 순회 정리

방식	특징	비고
for loop	- 배열의 인덱스를 이용하여 각 요소에 접근 - break, continue 사용 가능	
for ... of	- 배열 요소에 바로 접근 가능 - break, continue 사용 가능	
forEach()	- 간결하고 가독성이 높음 - callback 함수를 이용하여 각 요소를 조작하기 용이 - break, continue 사용 불가	사용 권장

<표2_Javascript_Basic syntax 02_배열 순회 정리>

```
const names = ['Alice', 'Bella', 'Cathy']

// for loop
for (let idx = 0; idx < names.length; idx++) {
  console.log(names[idx])
}

// for...of
for (const name of names) {
  console.log(name)
}

// forEach
names.forEach((name) => {
  console.log(name)
})
```

기타 Array Helper Methods

- MDN 문서를 참고해 사용해보기

메서드	역할
filter	콜백 함수의 반환 값이 참이 요소들만 모아서 새로운 배열을 반환
find	콜백 함수의 반환 값이 참이면 해당 요소를 반환
some	배열의 요소 중 적어도 하나라도 콜백 함수를 통과하면 true를 반환하며 즉시 배열 순회 중지 반면에 모두 통과하지 못하면 false를 반환
every	배열의 모든 요소가 콜백 함수를 통과하면 true를 반환 반면에 하나라도 통과하지 못하면 즉시 false를 반환하고 배열 순회 중지

<표3_Javascript_Basic syntax 02_기타 Array Helper Methods>

배열 with '전개 구문'

배열 with '전개 구문'

- '...' 은 배열의 괄호를 없애고 내용물만 꺼내기 때문에, 배열을 합치거나 중간에 삽입할 때 유용
- 전개 구문은 항상 새로운 배열을 만듭니다. 원본 배열은 전혀 변경되지 않음
- 배열 안의 객체는 데이터가 아닌, 주소값만 복사됩니다. 복사본의 객체를 수정하면 원본도 바뀜

```
let parts = ['어깨', '무릎']  
let lyrics = ['머리', ...parts, '발']  
  
console.log(lyrics) // [ '머리', '어깨', '무릎', '  
발' ]
```

참고

클래스

클래스

클래스

class

객체를 생성하기 위한 템플릿

- 객체의 속성, 메서드를 정의하는 청사진 역할
- '붕어빵'을 하나 만들 때마다 손으로 직접 모양을 빚는다고 상상해보세요.
100개를 만들려면 100번의 고된 수작업이 필요하겠죠?
객체를 하나씩 선언하는 것은 이와 같습니다. 만약 우리에게 '붕어빵 틀'이 있다면 어떨까요?
이 '붕어빵 틀'이 바로 "클래스(class)" 입니다.

클래스 기본 문법

1. class 키워드

- 객체의 '설계도'인 클래스를 정의하기 위해 사용하는 예약어
- 호이스팅 되지만, 선언 전에 접근하면 에러가 발생합니다. (TDZ)

2. 클래스 이름

- 일반적으로 파스칼 케이스(PascalCase)로 작성
- 함수처럼 이름을 생략한 '익명 클래스 표현식'으로 작성하는 것도 가능

3. 생성자 메서드(constructor())

- new로 객체 생성 시 자동으로 호출되며, 속성 값의 초기 설정을 담당
- 'constructor'라는 이름을 가진 메서드가 단 하나만 존재할 수 있음



호이스팅 :

코드를 실행하기 전에 함수, 변수, 클래스 등 맨 위로 끌어올리는 것처럼 보이는 현상

<개념5_Javascript_Basic syntax 02_호이스팅>



TDZ (Temporal Dead Zone):

let, const 선언 전, 변수 접근을 막는 일시적 사각 지대

<개념6_Javascript_Basic syntax 02_TDZ(Temporal Dead Zone)>

```
class MyClass {
  // 여러 메서드를 정의할 수 있음
  constructor() { ... }
  method1() { ... }
  method2() { ... }
  method3() { ... }
  ...
}
```

클래스 특징

- ES6에서 도입
- 생성자 함수를 사용하던 기존의 객체 생성 방식을 더 명확하고 객체 지향적으로 표현하기 위해 도입
- 그래서 클래스는 내부적으로 생성자 함수를 기반으로 동작함

// 생성자 함수

```
function Member(name, age) {  
  this.name = name  
  this.age = age  
  this.sayHi = function () {  
    console.log(`Hi, I am ${this.name}`)  
  }  
}
```

// 클래스

```
class Member {  
  constructor(name, age) {  
    this.name = name  
    this.age = age  
  }  
  sayHi() {  
    console.log(`Hi, I am ${this.name}`)  
  }  
}
```


클래스 활용

- new 키워드는 새 객체를 만들고, constructor를 호출하여 초기 속성 값을 설정
- 메서드 안의 this는 메서드를 호출한 member3 자신을 가리킴

```
class Member {  
  constructor(name, age) {  
    this.name = name  
    this.age = age  
  }  
  sayHi() {  
    console.log(`Hi, I am ${this.name}`)  
  }  
}  
  
const member3 = new Member('Alice', 20)  
  
console.log(member3) // Member { name: 'Alice', age: 20 }  
console.log(member3.name) // Alice  
member3.sayHi() // Hi I am Alice
```


new 연산자

'new' 연산자

```
const instance = new ClassName(arg1, arg2)
```

클래스나 생성자 함수를 사용하여 새로운 객체를 생성

- 클래스의 constructor()는 new 연산자에 의해 자동으로 호출되며 특별한 절차 없이 객체를 초기화 할 수 있음
- new 없이 클래스를 호출하면 TypeError 발생

콜백 함수의 이점

콜백 함수 구조를 사용하는 이유 (1/2)

- 함수 유연성 측면

- 함수를 호출하는 코드에서 콜백 함수의 동작을 자유롭게 변경할 수 있음
- map 함수는 동일하지만, 어떤 '콜백 함수'를 전달하느냐에 따라 결과가 달라짐

```
const numbers = [1, 2, 3, 4];  
// 콜백 함수 1: 각 요소를 두 배로 만드는 함수  
const double = function (number) {  
  return number * 2;  
};  
// 콜백 함수 2: 각 요소를 제곱하는 함수  
const square = function (number) {  
  return number * number;  
};  
// 1. double 콜백을 사용  
const doubledNumbers = numbers.map(double);  
console.log(doubledNumbers); // 출력: [2, 4, 6, 8]  
  
// 2. square 콜백을 사용  
const squaredNumbers = numbers.map(square);  
console.log(squaredNumbers); // 출력: [1, 4, 9, 16]
```

콜백 함수 구조를 사용하는 이유 (2/2)

• 비동기적 측면

- setTimeout 함수는 콜백 함수를 인자로 받아 일정 시간이 지난 후에 실행됨
 - 이때, setTimeout 함수는 비동기적으로 콜백 함수를 실행하므로, 다른 코드의 실행을 방해하지 않음
- ※ 비동기 JavaScript에서 자세히 진행 예정

```
console.log('a')  
  
setTimeout(() => {  
  console.log('b')  
}, 3000)  
  
console.log('c')
```



```
// a  
// c  
// b
```


forEach에서 break 사용하기

forEach에서는 break 사용하기

- forEach에서는 break 키워드를 사용할 수 없음
- 대신 some과 every의 특징을 활용해 마치 break를 사용하는 것처럼 활용 할 수 있음

// some 동작 예시

```
const array = [1, 2, 3, 4, 5]
```

```
const isEvenNumber = array.some(function (element) {  
  return element % 2 === 0  
})
```

```
console.log(isEvenNumber) // true
```

// every 동작 예시

```
const array = [1, 2, 3, 4, 5]
```

```
const isAllEvenNumber = array.every(function (element) {  
  return element % 2 === 0  
})
```

```
console.log(isAllEvenNumber) // false
```

forEach에서 break 하는 대안 (1/2)

- some을 활용한 예시
 - 콜백 함수가 true를 반환하면 즉시 순회를 중단하는 특징을 활용

```
const names = ['Alice', 'Bella', 'Cathy']

names.some(function (name) {
  console.log(name) // Alice, Bella
  if (name === 'Bella') {
    return true
  }
  return false
})
```

forEach에서 break 하는 대안 (2/2)

- every를 활용한 예시
 - 콜백 함수가 false를 반환하면 즉시 순회를 중단하는 특징을 활용

```
const names = ['Alice', 'Bella', 'Cathy']

names.every(function (name) {
  console.log(name) // Alice, Bella
  if (name === 'Bella') {
    return false
  }
  return true
})
```


배열은 객체다

배열은 객체다

- 배열도 키와 속성들을 담고 있는 참조 타입의 객체
- 배열의 요소를 대괄호 접근법을 사용해 접근하는 건 객체 문법과 같음
 - 배열의 키는 숫자
- 숫자형 키를 사용해 객체의 기본 기능 외로 “순서가 있는 컬렉션”을 제어하는 특별한 메서드를 제공
- 배열은 인덱스를 키로 가지며 length 속성을 갖는 특수한 객체

```
const numbers = [1, 2, 3]
console.log(Object.getOwnPropertyDescriptors(numbers))
```

```
▼ {0: {...}, 1: {...}, 2: {...}, length: {...}} ⓘ
  ▶ 0: {value: 'Alice', writable: true, enumerable: true, configurable: true}
  ▶ 1: {value: 'Bella', writable: true, enumerable: true, configurable: true}
  ▶ 2: {value: 'Cathy', writable: true, enumerable: true, configurable: true}
  ▶ length: {value: 3, writable: true, enumerable: false, configurable: false}
  ▶ [[Prototype]]: Object
```

 객체

- 3080. 함수와 객체 활용하기
- 2972. 객체와 메서드
- 2973. 객체 활용하기

 Array helper method

- 3079. 다양한 함수 표현
- 2968. 배열 활용하기
- 2971. 배열 순회2

Q 각 문제에 올바른 답안을 생각해봅시다

1 this 키워드에 대한 설명으로 올바른 것은?

- a) 항상 전역 객체(window)를 가리킨다.
- b) 함수가 선언될 때 가리키는 대상이 정해진다.
- c) 메서드를 호출한 객체를 가리킨다.
- d) 화살표 함수는 자신만의 this를 가진다.

2 키와 값의 변수명이 같을 때 사용하는 문법은?

- a) 단축 속성
- b) 계산된 속성
- c) 구조 분해 할당
- d) 전개 구문

3 객체의 속성을 분해하여 변수에 할당하는 문법은?

- a) 단축 속성
- b) 계산된 속성
- c) 구조 분해 할당
- d) Optional Chaining

4 객체를 JSON 형식의 문자열로 변환하는 메서드는?

- a) JSON.parse()
- b) JSON.stringify()
- c) Object.toJSON()
- d) Object.toString()

Q 각 문제에 올바른 답안을 생각해봅시다

5 배열의 가장 마지막에 요소를 추가하는 메서드는?

- a) pop()
- b) shift()
- c) unshift()
- d) push()

6 배열의 가장 앞에서 요소를 제거하는 메서드는?

- a) pop()
- b) shift()
- c) unshift()
- d) push()

7 콜백 함수에 대한 설명으로 가장 적절한 것은?

- a) 함수가 자기 자신을 다시 호출하는 함수
- b) 다른 함수의 인자로 전달되는 함수
- c) 여러 개의 함수를 하나로 묶는 함수
- d) 반드시 비동기적으로 작동하는 함수

8 배열의 각 요소에 대해 함수를 실행하지만, 아무것도 반환하지 않는 메서드는?

- a) map()
- b) filter()
- c) forEach()
- d) reduce()

Q 각 문제에 올바른 답안을 생각해봅시다

9 배열의 모든 요소에 함수를 적용한 결과로 새로운 배열을 만드는 메서드는?

- a) map()
- b) forEach()
- c) for...of
- d) Object.keys()

11 Optional Chaining (?.)을 사용하는 주된 목적은?

- a) 속성이 없으면 에러를 발생시키기 위해
- b) 중첩 객체의 속성을 에러 없이 안전하게 접근하기 위해
- c) 여러 함수를 연달아 호출하기 위해
- d) 중첩된 객체의 속성을 삭제하기 위해

10 객체의 모든 키(key)를 배열로 반환하는 메서드는?

- a) Object.keys()
- b) Object.values()
- c) Object.entries()
- d) object.keys()

12 클래스로부터 새로운 객체 인스턴스를 생성할 때 사용하는 키워드는?

- a) create
- b) instance
- c) new
- d) this

확인 문제



정답 및 해설

1. c) 메서드를 호출한 객체를 가리킨다. 2. a) 단축 속성 3. c) 구조 분해 할당 4. b) JSON.stringify() 5. d) push() 6. b) shift()
7. b) 다른 함수의 인자로 전달되는 함수 8. c) forEach() 9. a) map() 10. a) Object.keys()

1. this는 함수를 호출하는 방법에 따라 결정되며, 메서드 호출 시에는 해당 메서드를 호출한 객체를 가리킵니다.
2. 객체를 정의할 때, 키 이름과 값으로 쓰이는 변수 이름이 같다면 하나로 축약하여 사용할 수 있습니다.
3. 객체의 속성을 여러 변수로 쉽게 추출할 수 있게 해주는 문법으로, 코드의 가독성을 높여줍니다.
4. JSON.stringify()는 JavaScript 값이나 객체를 JSON 문자열로 변환하는 역할을 합니다.
5. push() 메서드는 배열의 끝에 하나 이상의 요소를 추가하고, 배열의 새로운 길이를 반환합니다.
6. shift() 메서드는 배열에서 첫 번째 요소를 제거하고, 제거된 그 요소를 반환합니다.

7. 다른 함수의 인자로 전달되어, 특정 작업이 끝난 후 외부 함수 내에서 호출되는 함수입니다.
8. forEach()는 각 요소에 대해 주어진 함수를 실행하지만, 항상 undefined를 반환합니다.
9. map()은 배열의 모든 요소에 함수를 적용하고, 그 결과들을 모아 새로운 배열로 반환합니다.
10. Object.keys(obj)는 주어진 객체가 가진 속성들의 이름(키)을 문자열 배열로 반환합니다.

확인 문제



정답 및 해설

11. b) 중첩 객체의 속성을 에러 없이 안전하게 접근하기 위해 12. c) new

- 11. 참조가 null 또는 undefined일 때 에러 대신 undefined를 반환하여 코드를 간결하게 만듭니다.
- 12. new 연산자는 클래스의 생성자(constructor)를 호출하여 새로운 객체 인스턴스를 생성합니다.

핵심 키워드

개념	설명	예시
객체 (Object)	키-값 쌍의 데이터 집합	{ key: 'value' }
메서드 (Method)	객체 속성에 정의된 함수	user.greeting()
this 키워드	함수를 호출한 객체를 가리킴	console.log(this.name)
구조 분해 할당	객체 속성을 변수에 할당하는 문법	const { name } = user
JSON	데이터를 표현하는 문자열 형식	JSON.stringify(obj)
콜백 함수	다른 함수에 인자로 전달되는 함수	numbers.forEach(myFunc)
forEach 메서드	배열의 모든 요소를 순회	names.forEach((name) => {})

〈표4_Javascript_Basic syntax 02_핵심키워드〉

활동 정리

오늘 공부한 내용 요약 및 정리

- 객체 (Object)
 - 객체는 키(key)와 값(value)으로 구성된 속성(property)들의 집합을 저장하는 자료형
 - 점(.)이나 대괄호([])를 사용해 속성에 접근, 추가, 수정, 삭제 가능
 - 메서드와 'this'
 - 메서드는 객체의 속성에 할당된 함수를 의미
 - this 키워드는 메서드를 호출한 객체
 - 일반 함수 안에서 this는 전역 객체(window)를 가리키지만, 화살표 함수는 자신만의 this를 갖지 않고 상위 스코프의 this를 참조
 - 이 특징 때문에 객체 메서드 내의 중첩된 함수에서 this를 다룰 때 화살표 함수가 유용함

활동 정리

오늘 공부한 내용

요약

및

정리

• 추가 객체 문법

• 구조 분해 할당

- `{ a, b } = obj`와 같이 객체의 속성을 분해하여 변수에 쉽게 할당하는 문법

• 전개 구문

- `const newObj = { ...oldObj }`처럼 객체의 속성을 펼쳐서 새로운 객체를 만들 때 사용(얕은 복사)

• Optional Chaining (`?.`):

- `user.address?.street`처럼 중첩된 객체의 속성을 에러 없이 안전하게 접근할 때 사용
- 중간 속성이 `null`이나 `undefined`이면 에러 대신 `undefined`를 반환함

활동 정리

오늘 공부한 내용 요약 및 정리

- JSON (JavaScript Object Notation)
 - JSON은 데이터를 표현하기 위한 문자열 형식
 - JavaScript 객체와 구조는 유사하지만, 전체가 하나의 문자열이라는 점이 차이
 - `JSON.stringify()`는 JavaScript 객체를 JSON 문자열로 변환
 - `JSON.parse()`는 JSON 문자열을 JavaScript 객체로 변환

활동 정리

오늘 공부한 내용 요약 및 정리

- 배열 (Array)
 - 배열은 순서가 있는 데이터의 집합을 저장하는 자료구조
 - 배열 메서드
 - push()와 pop()은 배열의 끝에서 요소를 추가하거나 제거
 - unshift()와 shift()는 배열의 앞에서 요소를 추가하거나 제거
 - 이 메서드들은 모두 원본 배열을 직접 수정

활동 정리

오늘 공부한 내용 요약 및 정리

• 배열 (Array) - Array Helper Methods

- 배열을 순회하며 특정 작업을 수행하는 고차 함수들로, 콜백 함수를 인자로 받음
- 콜백 함수
 - 다른 함수의 인자로 전달되어, 외부 함수 내에서 호출되는 함수
- `forEach()`
 - 배열의 각 요소에 대해 콜백 함수를 실행함 반환 값은 없음
- `map()`
 - 배열의 각 요소에 대해 콜백 함수를 실행하고, 그 결과를 모아 새로운 배열을 반환
 - 원본 배열은 변경되지 않음

활동 정리

"여러 사람의 데이터를 다루려면 무엇부터 생각해야 할까요?"

1. 먼저 한 사람의 이름, 나이, 직업을 어떻게 묶을 수 있을까요? (**객체**)
2. 여러 사람의 데이터를 목록으로 관리하려면 어떻게 해야 할까요? (**배열**)
3. 목록의 모든 사람에게 똑같은 작업을 시키려면 어떻게 해야 할까요? (**배열 헬퍼 메서드**)

```
// 한 사람의 데이터를 어떻게 묶을까? -> 객체
const user1 = { name: 'jayden', age: 20 };
const user2 = { name: 'harry', age: 21 };

// 여러 사람을 어떻게 목록으로 만들까? -> 배열
const users = [user1, user2];

// 목록 전체에 같은 작업을 하려면? -> 배열 헬퍼 메서드
const userNames = users.map(user => user.name); // ['jayden', 'harry']
```

감사합니다

